

# UltraHPM API Gateway - Complete Configuration

## ▮ Step 1: Fix API Gateway POM

Replace your `api-gateway/pom.xml` with this corrected version:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <!-- IMPORTANT: Inherit from parent POM, not Spring Boot directly -->
  <parent>
    <groupId>com.ultrahpm</groupId>
    <artifactId>ultra-high-performance-microservices</artifactId>
    <version>1.0.0-SNAPSHOT</version>
    <relativePath>../pom.xml</relativePath>
  </parent>

  <artifactId>api-gateway</artifactId>
  <name>UltraHPM API Gateway</name>
  <description>High-performance Spring Cloud Gateway</description>

  <dependencies>
    <!-- Spring Cloud Gateway (WebFlux-based - REACTIVE) -->
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-gateway</artifactId>
    </dependency>

    <!-- Service Discovery -->
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
    </dependency>

    <!-- Circuit Breaker & Resilience -->
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
    </dependency>

    <!-- Monitoring & Health -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
  </dependencies>
</project>
```

```

<!-- Metrics -->
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>

<!-- Redis for Rate Limiting -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis-reactive</artifactId>
</dependency>

<!-- JSON Processing -->
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
</dependency>

<!-- Testing -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>

<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-contract-stub-runner</artifactId>
    <scope>test</scope>
</dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <image>
                    <name>ultrahpm/${project.artifactId}:${project.version}</name>
                </image>
            </configuration>
        </plugin>
    </plugins>
</build>
</project>

```

## ▮ Step 2: Main Application Class

**Update** api-gateway/src/main/java/com/ultrahpm/apigateway/ApiGatewayApplication.java:

```

package com.ultrahpm.apigateway;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

```

```

import org.springframework.cloud.client.discovery.EnableDiscoveryClient;

@SpringBootApplication
@EnableDiscoveryClient
public class ApiGatewayApplication {

    public static void main(String[] args) {
        System.out.println("🚀 Starting UltraHPM API Gateway...");
        System.out.println("🚀 High-Performance Reactive Gateway with Circuit Breakers");
        SpringApplication.run(ApiGatewayApplication.class, args);
        System.out.println("✅ UltraHPM API Gateway is Ready!");
    }
}

```

### ⚙️ Step 3: Gateway Configuration Class

**Create** `api-gateway/src/main/java/com/ultrahpm/apigateway/config/GatewayConfig.java`:

```

package com.ultrahpm.apigateway.config;

import org.springframework.cloud.gateway.filter.ratelimit.KeyResolver;
import org.springframework.cloud.gateway.route.RouteLocator;
import org.springframework.cloud.gateway.route.builder.RouteLocatorBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import reactor.core.publisher.Mono;

@Configuration
public class GatewayConfig {

    @Bean
    public RouteLocator customRouteLocator(RouteLocatorBuilder builder) {
        return builder.routes()

            // Product Service Routes - GraphQL + REST
            .route("product-service-graphql", r -> r
                .path("/graphql", "/graphiql")
                .uri("lb://product-service"))

            .route("product-service-rest", r -> r
                .path("/api/products/**")
                .filters(f -> f
                    .stripPrefix(1)
                    .circuitBreaker(config -> config
                        .setName("product-service-cb")
                        .setFallbackUri("forward:/fallback/products"))
                    .requestRateLimiter(config -> config
                        .setRateLimiter(redisRateLimiter())
                        .setKeyResolver(userKeyResolver())))
                .uri("lb://product-service"))

            // Order Service Routes - gRPC + REST
            .route("order-service", r -> r
                .path("/api/orders/**")
                .filters(f -> f

```

```

        .stripPrefix(1)
        .circuitBreaker(config -> config
            .setName("order-service-cb")
            .setFallbackUri("forward:/fallback/orders")))
        .uri("lb://order-service"))

// Payment Service Routes
.route("payment-service", r -> r
    .path("/api/payments/**")
    .filters(f -> f
        .stripPrefix(1)
        .circuitBreaker(config -> config
            .setName("payment-service-cb")
            .setFallbackUri("forward:/fallback/payments")))
    .uri("lb://payment-service"))

// Notification Service Routes
.route("notification-service", r -> r
    .path("/api/notifications/**")
    .filters(f -> f
        .stripPrefix(1)
        .circuitBreaker(config -> config
            .setName("notification-service-cb")
            .setFallbackUri("forward:/fallback/notifications")))
    .uri("lb://notification-service"))

    .build();
}

@Bean
public KeyResolver userKeyResolver() {
    return exchange -> Mono.just("user-" +
        exchange.getRequest().getRemoteAddress().getAddress().getHostAddress());
}

@Bean
public org.springframework.cloud.gateway.filter.ratelimit.RedisRateLimiter redisRateLimiter() {
    return new org.springframework.cloud.gateway.filter.ratelimit.RedisRateLimiter(1000);
}
}

```

## ▮ Step 4: Fallback Controller

### Create api-

gateway/src/main/java/com/ultrahpm/apigateway/controller/FallbackController.java:

```

package com.ultrahpm.apigateway.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import reactor.core.publisher.Mono;

```

```

import java.time.LocalDateTime;
import java.util.Map;

@RestController
@RequestMapping("/fallback")
public class FallbackController {

    @GetMapping("/products")
    public Mono<ResponseEntity<Map<String, Object>>> productsFallback() {
        return Mono.just(ResponseEntity.ok(Map.of(
            "message", "Product service is currently unavailable",
            "status", "fallback",
            "timestamp", LocalDateTime.now(),
            "service", "product-service"
        )));
    }

    @GetMapping("/orders")
    public Mono<ResponseEntity<Map<String, Object>>> ordersFallback() {
        return Mono.just(ResponseEntity.ok(Map.of(
            "message", "Order service is currently unavailable",
            "status", "fallback",
            "timestamp", LocalDateTime.now(),
            "service", "order-service"
        )));
    }

    @GetMapping("/payments")
    public Mono<ResponseEntity<Map<String, Object>>> paymentsFallback() {
        return Mono.just(ResponseEntity.ok(Map.of(
            "message", "Payment service is currently unavailable",
            "status", "fallback",
            "timestamp", LocalDateTime.now(),
            "service", "payment-service"
        )));
    }

    @GetMapping("/notifications")
    public Mono<ResponseEntity<Map<String, Object>>> notificationsFallback() {
        return Mono.just(ResponseEntity.ok(Map.of(
            "message", "Notification service is currently unavailable",
            "status", "fallback",
            "timestamp", LocalDateTime.now(),
            "service", "notification-service"
        )));
    }
}

```

## ▮ Step 5: Application Configuration

**Create** `api-gateway/src/main/resources/application.yml`:

```

# UltraHPM API Gateway Configuration
spring:
  application:

```

```
name: api-gateway

# Redis Configuration for Rate Limiting
data:
  redis:
    host: localhost
    port: 6379
    timeout: 2000ms
    lettuce:
      pool:
        max-active: 20
        max-idle: 8
        min-idle: 2

# Cloud Gateway Configuration
cloud:
  gateway:
    # Enable service discovery
    discovery:
      locator:
        enabled: true
        lower-case-service-id: true

    # Global filters
    default-filters:
      - DedupeResponseHeader=Access-Control-Allow-Credentials Access-Control-Allow-Origin

# CORS Configuration
globalcors:
  corsConfigurations:
    '[/*]':
      allowedOrigins:
        - "http://localhost:3000"
        - "http://localhost:3001"
        - "http://localhost:8081"
      allowedMethods: "*"
      allowedHeaders: "*"
      allowCredentials: true

# Circuit Breaker Configuration
circuitbreaker:
  resilience4j:
    enabled: true

# Server Configuration
server:
  port: 8080
  http2:
    enabled: true
  compression:
    enabled: true

# Eureka Configuration
eureka:
  client:
    service-url:
```

```

    defaultZone: http://localhost:8761/eureka/
    register-with-eureka: true
    fetch-registry: true
    registry-fetch-interval-seconds: 5
instance:
    prefer-ip-address: true
    lease-renewal-interval-in-seconds: 10
    lease-expiration-duration-in-seconds: 30
    health-check-url-path: /actuator/health

# Management & Monitoring
management:
    endpoints:
        web:
            exposure:
                include: health,info,prometheus,gateway,metrics,circuitbreakers
    endpoint:
        health:
            show-details: always
            show-components: always
        gateway:
            enabled: true
    metrics:
        tags:
            application: ${spring.application.name}
            environment: local
        distribution:
            percentiles-histogram:
                http.server.requests: true
            percentiles:
                http.server.requests: 0.5, 0.9, 0.95, 0.99

# Resilience4j Configuration
resilience4j:
    circuitbreaker:
        configs:
            default:
                slidingWindowSize: 50
                minimumNumberOfCalls: 20
                failureRateThreshold: 50
                waitDurationInOpenState: 10s
                permittedNumberOfCallsInHalfOpenState: 5
                slowCallRateThreshold: 50
                slowCallDurationThreshold: 2s
    instances:
        product-service-cb:
            baseConfig: default
        order-service-cb:
            baseConfig: default
        payment-service-cb:
            baseConfig: default
        notification-service-cb:
            baseConfig: default

timelimiter:
    instances:

```

```

        product-service-cb:
            timeoutDuration: 3s
        order-service-cb:
            timeoutDuration: 3s
        payment-service-cb:
            timeoutDuration: 5s
        notification-service-cb:
            timeoutDuration: 2s

# Logging Configuration
logging:
    level:
        org.springframework.cloud.gateway: INFO
        com.ultrahpm.apigateway: DEBUG
        io.github.resilience4j: DEBUG
    pattern:
        console: "%clr(%d{HH:mm:ss.SSS}){faint} %clr(${LOG_LEVEL_PATTERN:%5p}) %clr([%t]){fai

# Custom Properties
ultrahpm:
    gateway:
        rate-limit:
            default-requests-per-second: 100
            default-burst-capacity: 200
        circuit-breaker:
            failure-threshold: 50
            slow-call-threshold: 2000

```

## ▮ Step 6: Testing Configuration

**Create** `api-gateway/src/test/java/com/ultrahpm/apigateway/ApiGatewayApplicationTests.java`:

```

package com.ultrahpm.apigateway;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.TestPropertySource;

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@TestPropertySource(properties = {
    "eureka.client.enabled=false",
    "spring.data.redis.host=localhost",
    "spring.data.redis.port=6379"
})
class ApiGatewayApplicationTests {

    @Test
    void contextLoads() {
        // Test that Spring context loads successfully
    }
}

```



## ▮ Step 7: Test Your API Gateway

### 1. Start Infrastructure First

```
# Create infrastructure directory
mkdir -p infrastructure/docker

# Create basic docker-compose.yml (minimal for testing)
```

#### infrastructure/docker/docker-compose.yml:

```
version: '3.8'
services:
  eureka:
    image: steeltoeoss/eureka-server:3.1.3
    container_name: ultrahpm-eureka
    ports:
      - "8761:8761"
    environment:
      EUREKA_CLIENT_REGISTER_WITH_EUREKA: false
      EUREKA_CLIENT_FETCH_REGISTRY: false

  redis:
    image: redis:7-alpine
    container_name: ultrahpm-redis
    ports:
      - "6379:6379"
```

```
cd infrastructure/docker
docker-compose up -d
```

### 2. Run API Gateway

```
cd api-gateway
mvn spring-boot:run
```

### 3. Test Endpoints

```
# Health check
curl http://localhost:8080/actuator/health

# Gateway routes
curl http://localhost:8080/actuator/gateway/routes

# Test fallback
curl http://localhost:8080/api/products
```

## 4. Verify Service Registration

Visit: <http://localhost:8761> (should show API-GATEWAY registered)

### ✓ Success Indicators

- ✓ **Application starts** without errors
- ✓ **Registers with Eureka** (visible at <http://localhost:8761>)
- ✓ **Health endpoint** returns UP status
- ✓ **Gateway routes** are visible in actuator
- ✓ **Fallback endpoints** return proper JSON responses

### ▮ Next Steps After Gateway Works

1. ✓ **Create Product Service** (WebFlux + GraphQL)
2. ✓ **Test routing** through gateway
3. ✓ **Add remaining services**
4. ✓ **Load testing** and performance tuning

**Is your API Gateway working now? Let me know the results and we'll move to the Product Service next!**